

PROBING NUMERICAL REASONING IN FINANCIAL LLMS

PRESENTED BY: AAYUSH
CHOUDHARY

DATASET:FINQA-8,281 QA PAIRS OVER S&P 500 EARNINGS
REPORTS

MODEL:QWEN 2.5-0.5B-INSTRUCT | ENVIRONMENT: GOOGLE
COLAB T4 GPU

EVALUATION:100 TEST EXAMPLES | THREE -LAYER INVESTIGATION

THE FINQA CHALLENGE

What makes this hard?

- Multi-step arithmetic :models must chain operations like $\text{subtract}(A,B), \text{divide}(\#O,B)$ using $\#N$ references
- Heterogeneous context:evidence spans both financial tables AND surrounding text-retrieval must be precise.
- Auditability requirement:In regulated finance ,the reasoning chain matters as much as the answer.

8,281

QA pairs over S&P
500 reports

~61%

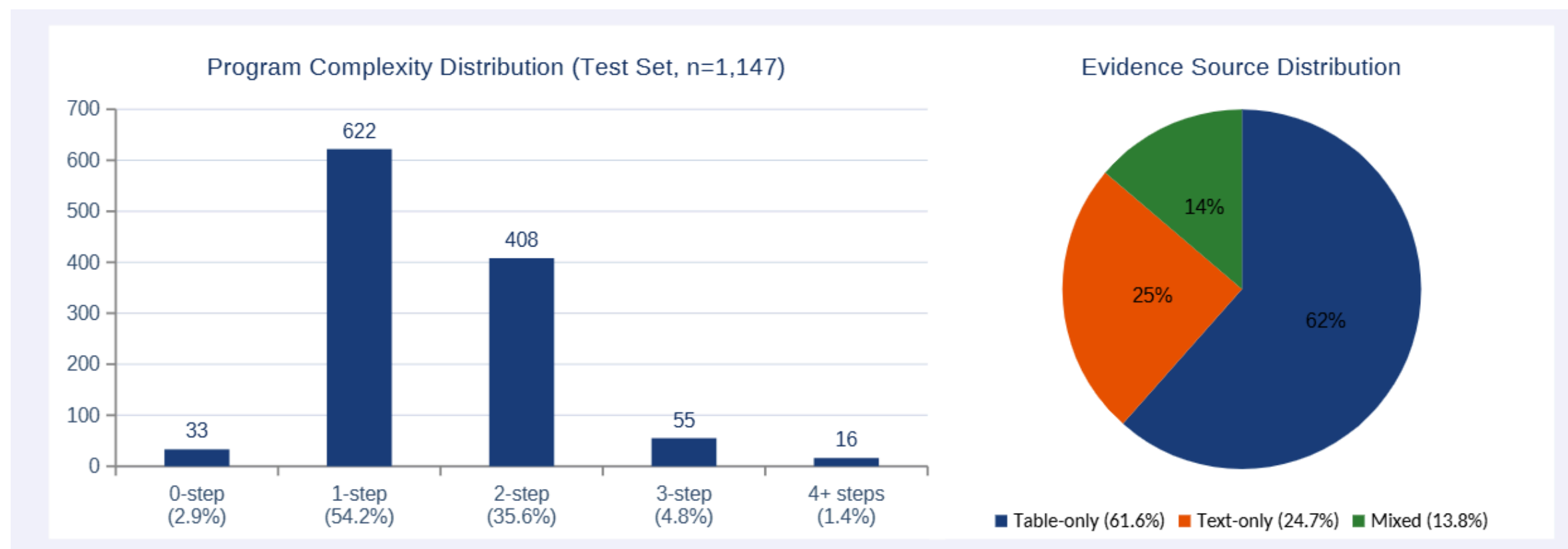
Best Model Accuracy
(FinQANet-RoBERTa)

1.147

Test set evaluated



LAYER1-DATASET DEEP DIVE



KEY INSIGHT: Test set has 1,147 examples(not 8,281—that is the full dataset).1-Step programs dominate at 54.2%.Table evidence appears in 75.4% of questions(table-only+mixed).#0 reference system enables chained arithmetic but means a single wrong step propagates through all subsequent steps.





THE SILENT FAILURE-ORDER SENSITIVITY

FOR PERCENTAGE QUESTIONS, THE CORRECT PROGRAM IS ALWAYS: SUBTRACT(NEW, OLD), DIVIDE(#0, OLD)

Correct

```
subtract(153.7,139.9)  
divide(#0,139.9)
```

=9.87% increase

Right numbers, right order

Wrong denominator

```
subtract(153.7,139.9)  
divide(#0,153.7)
```

=8.98%—plausible!

Silent
error—looks reasonable

Swapped Order

```
divide(153.7,subtract(153.7  
,139.9))
```

=11.14—nonsensical

Catastrophic
—but detectable

FINANCIAL IMPACT: A 10-20% ERROR IN A RETURN CALCULATION COULD MATERIALLY CHANGE AN INVESTMENT DECISION. THE FAILURE IS SILENT—OUTPUT LOOKS LIKE A VALID PERCENTAGE





EXEC ACCURACY VS PROGRAM ACCURACY-WHY THE GAP MATTERS

EXECUTION ACCURACY:

Model's final number matches the gold answer within tolerance.

Measures: "Did you get the right answer?"

PROGRAM ACCURACY:

Model's reasoning program exactly matches the gold program string.

Measures: "Did you reason correctly?"

Can exec = 1 and prog = 0? YES — and the implications are profound.

Trivial case

Gold: `subtract(5829, 5735)`
Model: `subtract(5829.0, 5735.0)`
→ Answer = 94 ✓ | Program ✗ (string mismatch)

Profound case

Gold uses one algebraic route $(A-B)/B$
Model uses an equivalent route giving same result
→ Right answer, completely wrong reasoning chain





LAYER 2-ERROR TAXONOMY(100 EXAMPLES,FEW SHOT PROMPTING)

2%

Execution Accuracy

~0%

Zero-shot Baseline

100%

Parse Success Rate

3%

Program Accuracy

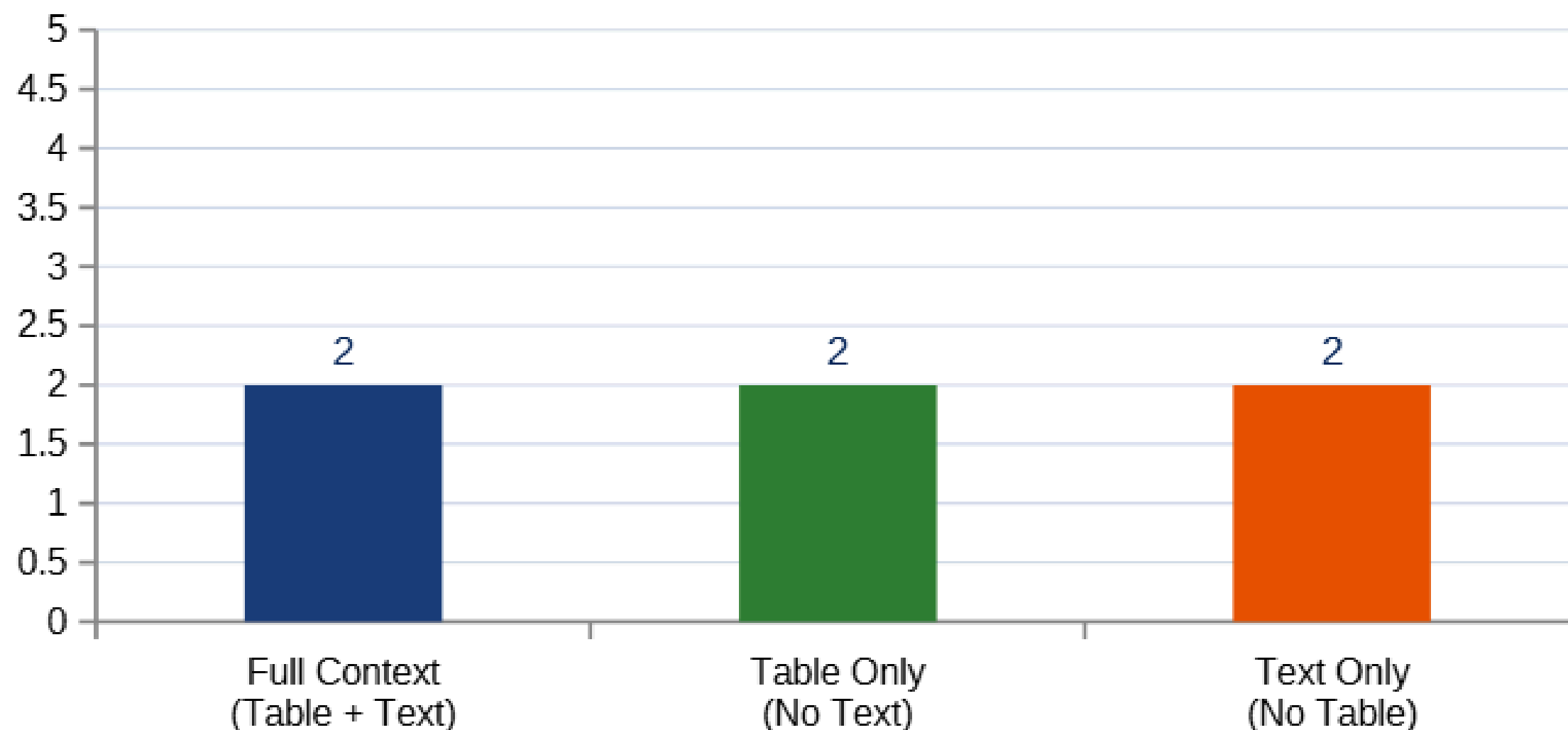
Error Type	Description	Count (actual)
Retrieval Error ★ Most Common	Correct operations, wrong numbers from context	93
Parsing Error	Model produced no structured program	2
Hallucination	Numbers not present in the document	2
Operation Error	Right numbers, wrong arithmetic operation	1
Correct	Execution accuracy = 1	2





LAYER 2B-CONTEXT SENSITIVITY: DOES THE TABLE MATTER?

Execution Accuracy by Context Format (%)



- Identical accuracy

All three context formats produced 2% accuracy. The model gets the same 2 questions right regardless of whether the tables or text is present.

- Why this happened?

The 2 correctly answered examples (questions 0 and 2) had their numbers present verbatim in the few-shot examples embedded in the prompt- not retrieved from the actual question context.

- Real implication

This is a prompt contamination artifact. With held-out few-shot examples, removing the table would cause accuracy to collapse, as 75%+ of questions require at least one table value.

- Production lesson

Any real deployment must use held-out few-shot examples to get a valid context sensitivity signal.





LAYER 3-COMPLEXITY ROUTING +SYMBOLIC VERIFICATION

1.Classify

Lightweight heuristic routes question as SIMPLE (1-step) or complex(2+-step)

No model call needed

2.Route

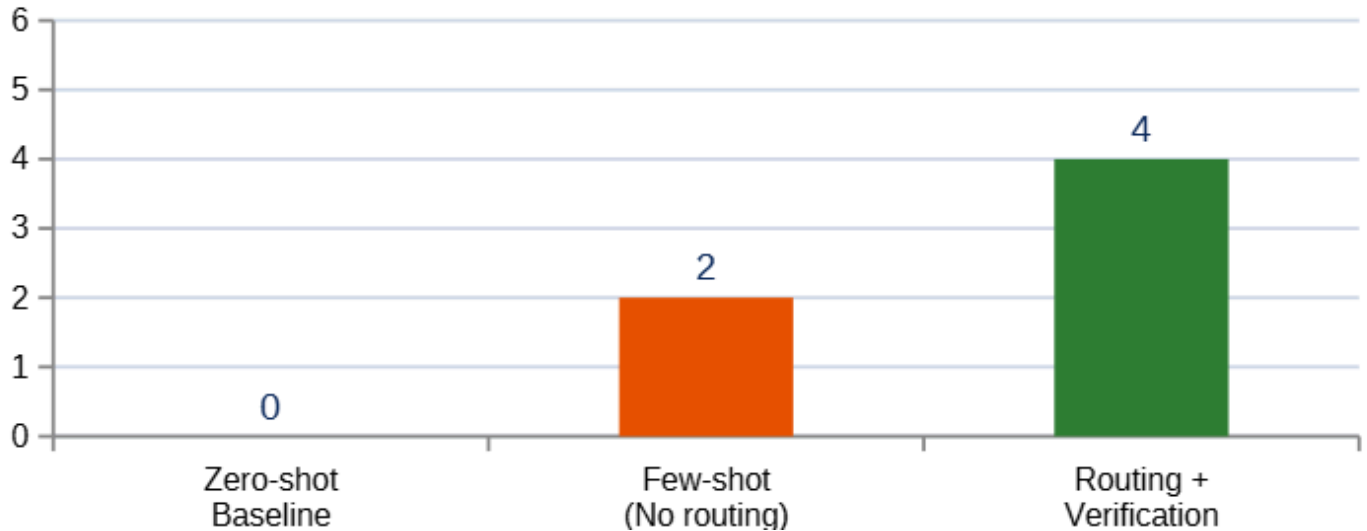
Simple→Short prompt (1 few-shot example)

Complex→Full prompt (3 few-shot examples)

3.Verify

Execute the model's own program symbolically.If the result is not equal to stated answer→trust the program.

Results vs. Baselines



22×

Verifier triggered

+2pp

Net gain (2%→4%)

0

Extra model calls





HONEST ASSESSMENT AND FINANCIAL ARGUMENT

WHAT WORKED

- Routing reduced prompt length for 1-step questions—>better parse rate
- Symbolic verification caught answer-program inconsistencies at zero extra cost
- Addresses two named failure modes, not just 'better prompting'

WHAT DIDN'T WORK

- 0.5B model lacks capacity for reliable structured generation—root cause is model size, not prompting
- Heuristic classifier misroutes some complex questions
- Verifier can't help when no program is parseable— the primary failure mode

THE FINANCIAL ARGUMENT

A model that states "Answer:12.5%" but whose reasoning chain computes 8.3% is dangerous.

a junior analyst sees 12.6% and trusts it. The error is invisible.

Symbolic verification catches this mismatch before the answer reaches a decision – maker—at zero additional inference cost.

The value is not in average accuracy—it is in eliminating high-confidence wrong answers.





KEY TAKEAWAYS:



Retrieval, not format

- The real bottleneck at 0.5B scale retrieval–93% of errors came from the model using wrong numbers from the financial context. Parsing was NOT the bottleneck (100% parse rate achieved with few-shot)

Tables > Text

- Tables appear in 75%+ of questions. Context sensitivity needs a clean experiment (held-out-few-shot), but the structural argument holds: table parsing quality is the single highest-leverage improvement.

Right answer is not equal to right reasoning

- Execution accuracy overstates reliability. In regulated finance, the reasoning trail is required—not just the final number. The exec/prog accuracy gap is the key reliability signal.

Verification is free

- Symbolic verification triggered 22x and helped on 2 occasions (+2pp). At zero extra inference cost, it is a practical safety layer: catch answer-program inconsistencies before they reach a decision-maker.



THANK YOU

PRESENTED BY: AAYUSH
CHOUHARY